

Embedded Systems - Fundamentals

System

A system is an **arrangement** in which **all its unit assemble to work together according to a set of rules**. It can also be defined as a way of working, organizing or doing one or many tasks according to a fixed plan. **For example, a watch is a time displaying system**. Its components follow a set of rules to show time. If one of its parts fails, the watch will stop working. So, we can say, in a system, all its **subcomponents depend on each other**.

Embedded System

- As its name suggests, **Embedded** means **something that is attached to another thing**.
- An Embedded System is a **specially designed computing** system that **combines a microprocessor, memory device, input-output devices, and dedicated software in a single unit**.
- An embedded system is **designed to perform a dedicated function** within a large electronic or mechanical system.
- An embedded system **can be an independent system** or it can be a **part of a large system**. It is referred to as an **embedded system** because it is **integrated (or embedded) as a part of a complex system or device**. For example, a **fire alarm** is an embedded system; it will sense only smoke.
- Embedded systems are being widely used in a **large range of applications** from **portable devices like smart watches, calculators, smartphones, etc.** to **large electrical or mechanical machines** like home appliances, robots, automation systems, vehicles, spacecrafts, and more.

Characteristics of an Embedded System

- **Performs Specific Task:** Embedded systems perform some specific function or tasks.
- **Low Cost:** The price of an embedded system is not so expensive.
- **Time Specific:** It performs the tasks within a certain time frame.
- **Low Power:** Embedded Systems don't require much power to operate.
- **High Efficiency:** The efficiency level of embedded systems is high.
- **Minimal User interface:** These systems require less user interface and are easy to use.
- **Less Human intervention:** Embedded systems require no human intervention or very less human intervention.

- **Highly Stable:** Embedded systems do not change frequently; mostly fixed maintaining stability.
- **High Reliability:** Embedded systems are reliable as they perform tasks consistently well.
- **Use microprocessors or microcontrollers:** Embedded systems use microprocessors or microcontrollers to design and use limited memory.
- **Memory:** It must have a memory, as its software usually embeds in ROM. It does not need any secondary memories in the computer.
- **Manufacturable:** The majority of embedded systems are compact and affordable to manufacture. They are based on the size and low complexity of the hardware.
- **Connected:** It must have connected peripherals to connect input and output devices.
- **HW-SW systems:** Software is used for more features and flexibility. Hardware is used for performance and security.

Components of Embedded System

An embedded system is a **microprocessor or microcontroller-based, software-driven, reliable, real-time control system** designed to perform a specific dedicated function. Unlike general-purpose computers, these systems are **constrained by power, size, and timing**. The major components of an embedded system are explained below.

(i) Hardware (The Physical Layer)

Hardware is responsible for the **physical execution of tasks**. It **converts electrical signals into actions**. Hardware components contain:

- **Processor (The Brain):**
 - Usually a **Microcontroller (MCU)** which integrates the CPU, memory, and I/O on one chip.
 - For high-performance tasks, a **Microprocessor (MPU)** is used.
- **Memory:**
 - **ROM/Flash:** Stores the **Firmware** (the permanent startup instructions).
 - **RAM:** Temporary workspace for real-time calculations.
- **I/O Peripherals:** These are the bridges to the outside world.
 - **Sensors:** Input physical data (e.g., temperature, speed).
 - **Actuators:** Output physical actions (e.g., moving a motor, flashing a light).

(ii) Software & Firmware (The Logic Layer)

This is the set of instructions that defines the system's "personality" and purpose.

- **Application Software:** The high-level code written to perform the specific task (e.g., the code that calculates the cook time in a microwave).
- **Firmware:** This is a specific type of software embedded directly into the hardware's non-volatile memory. It acts as the "glue" between the hardware and the application software.

(iii) The Operating System (The Management Layer)

While small-scale systems (like a simple flashlight) might not need an OS, complex systems use a **Real-Time Operating System (RTOS)**.

- **Role:** It supervises the application software and manages the processor's time.
- **Determinism:** The most critical feature of an RTOS is its ability to control **latencies**. It ensures a process runs exactly when scheduled, which is vital for safety-critical systems like automotive airbags or medical monitors.
- **Scheduling:** It follows a strict plan to ensure high-priority tasks are never delayed by low-priority ones.

Firmware Component

Firmware is a special type of software stored in non-volatile memory (ROM/Flash) that **directly controls hardware**. The **role of firmware in embedded systems** is:

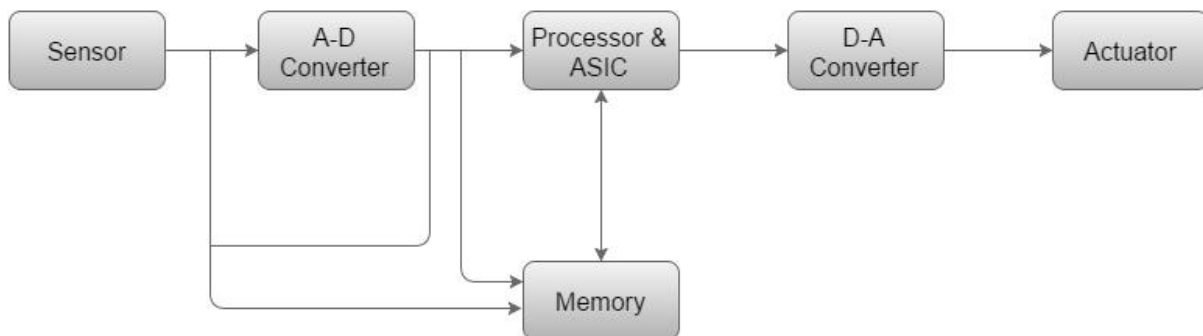
- Initializes hardware
- Loads application software
- Provides hardware abstraction

The **difference between firmware and software** in case of embedded system is given in below table.

Aspect	Firmware	Software
Storage	ROM/Flash	RAM/Storage
Purpose	Low-level control	High-level logic
Change Frequency	Rare	Frequent

Basic Structure of an Embedded System

The following illustration shows the basic structure of an embedded system.



- **Sensor**: It measures the physical quantity and converts it to an electrical signal which can be read by an observer or by any electronic instrument like an A2D converter. A sensor stores the measured quantity to the memory.
- **A-D Converter**: An analog-to-digital converter converts the analog signal sent by the sensor into a digital signal.
- **Processor & ASICs**: This is the central processing unit of the embedded system. It processes the input signals according to instructions to produce the results. Processors process the data to measure the output and store it to the memory.
- **Memory**: In an embedded system, memory chips are provided to store software and input-output data.
- **D-A Converter**: A digital-to-analog converter converts the digital data fed by the processor to analog signal.
- **Actuator**: An actuator compares the output given by the D-A Converter to the actual (expected) output stored in it and stores the approved output. These are the action taking components of the embedded system. They convert electric signal outputs of the processor into mechanical motion or action.

How Does an Embedded System Work?

An embedded system is a small, low-power, and low-cost computer designed to be used as a part of a large machine or system. Therefore, it works in the same way as an ordinary computer does. The **step-by-step working of an embedded system** is explained here:

- **Step 1**: It reads external inputs through sensors.
- **Step 2**: Convert these sensor inputs into processor readable format.

- **Step 3:** Processor executes instructions and converts the inputs into useful outputs.
- **Step 4:** Converts the processors outputs into actuator readable format.
- **Step 5:** Actuators read the outputs and perform actions.

Practical Example: The Washing Machine Cycle

To see these steps in action, let's look at a modern washing machine:

- **Input:** You press "Start" (User Input) and the water level sensor detects the drum is empty (Sensor Input).
- **Processing:** The Microcontroller reads the "Cotton Cycle" firmware and decides to open the water valve.
- **Output:** The **Actuator** opens the valve.
- **Feedback Loop:** Once the water level sensor hits the "Full" mark, the processor immediately sends a signal to close the valve and start the motor.
- **Real-Time:** The system constantly monitors for leaks or imbalances, stopping the motor instantly if an error is detected to prevent damage.

Classification of Embedded Systems

The **classification** of embedded systems can be **done on the basis of their functionality and performance** as given below.

Types of Embedded Systems Based on Functionality

Based on their functionality, embedded systems can be categorized as follows.

- **Mobile Embedded Systems:** Mobile embedded systems are small-sized and portable embedded systems. These embedded systems are commonly used in smartphones, laptops, computers, digital cameras, smart watches, etc.
- **Real-Time Embedded Systems:** Real-time embedded systems are designed to produce outputs in a definite time interval. These systems perform time-critical functions and are widely used in medical systems, industrial automation, traffic control systems, etc.
- **Networked Embedded Systems:** Networked embedded systems are designed to uses in network connected systems such as security systems, point-of-sale systems, remote monitoring systems, etc.
- **Standalone Embedded Systems:** Standalone embedded systems are designed to function independently without a host system or computer. Examples of this type of embedded systems include digital watches, MP3 players, calculators, etc.

Types of Embedded Systems Based on Performance Requirements

Based on their performance requirements, embedded systems can be classified into the following types.

- **Small-Scale Embedded Systems:** Small-scale embedded systems are designed using **8-bit or smaller** microprocessor or microcontroller. They are **limited in terms of memory and processing power**. However, these systems are cost-effective and are used in **traffic controllers, toys, smart TV remote controls, smart cards, etc.**
- **Medium-Scale Embedded Systems:** Medium-scale embedded systems use **16-bit or 32-bit** microprocessors or controllers. These systems are **relatively more complex and faster** than small-scale systems. They are commonly used in **smart home appliances, medical equipment, and automation systems.**
- **Sophisticated-Scale Embedded Systems:** Sophisticated-scale embedded systems are also referred to as **complex embedded systems**. They are designed using **64-bit or larger** microprocessors or controllers. These systems are **powerful in terms of memory and processing capabilities**. However, these are highly complex and more expensive. Common applications of these embedded systems include **advanced medical equipment, robotics, safety critical systems, etc.**

Embedded Systems Terminology

There are some very **important terms related to embedded systems**, which are briefly defined in this section. These definitions will be very helpful for students throughout this course.

- **Embedded Processor:** A microprocessor which is specially designed to use in embedded systems is referred to as an embedded processor. These processors are designed to provide excellent performance in terms of processing power, efficiency, and real-time operations.
- **Microcontroller:** A microcontroller, also referred to as microcontroller unit (MCU), is small-sized computer implemented on a single IC chip. A typical microcontroller consists of all essential components of a microcomputer such as microprocessor, memory unit, IO peripherals, and software. 8085 and 8086 are common examples of simple microcontrollers.
- **8051 Microcontroller:** 8051 Microcontroller is a single chip microcontroller developed by Intel Corporation in 1980 to use in embedded systems. It is also known as Intel MCS-51. The 8051 is an 8-bit microcontroller, as it can process 8-bits of data at a time. It is usual

employed in embedded system used in remote controls, robotics, and telecom applications.

- **Microprocessor:** In an embedded system, a microprocessor is the main processing unit that executes instructions and processes. It is an integrated circuit chip having data processing circuitry.
- **System-on-Chip (SoC):** System-on-a-Chip is an integrated circuit (IC) design that combines all the major components like processor, memory, input-output peripherals, etc. of an electronic device or system onto a single chip. It does not have any separate components mounted on a motherboard.
- **Architecture:** The fundamental structure and design of an embedded system that defines how it will handle the instructions and data and how its components will communicate is referred to as architecture of the embedded system. The commonly used architectures of embedded systems are Harvard architecture and Neumann architecture.
- **I/O Programming:** In embedded systems, I/O programming is referred to as the process of exchanging data and instructions between the embedded system and external devices like sensors, displays, motors, etc.
- **Assembly Language:** Assembly Language is a low-level language used in computer programming. In this programming language, the instructions are written by using abbreviated names equivalent to machine language codes. Assembly language is primarily used to write programs for microprocessors and microcontrollers.
- **Registers:** In embedded systems, a register is nothing but a small and high-speed temporary storage device used for holding instructions and data required for processing.
- **Register Bank / Stack:** In embedded systems, the register bank is nothing but a part of the RAM (Random Access Memory) used for storing program instructions. A register stack is a part in the RAM that temporarily stores information and access this information using a stack pointer register.
- **Instructions:** Instructions are the computer codes that the microprocessor of a computer or embedded system can understand and execute. These are generally written in binary language using 0s and 1s.
- **Addressing Modes:** The methods of specifying operands of instructions or locations of data in memory are referred to as addressing modes. Therefore, addressing modes define the rules for interpreting and manipulating the address fields of the operands of instructions before their actual execution. In embedded systems, immediate, direct, indirect, and indexed are some commonly used addressing modes.
- **Special Function Registers (SFRs):** Special function registers are those registers in an embedded system that monitor and control various aspects of the system operations. These registers are closely associated with some special functions and provide communication interfacing between the microprocessor and the peripherals.

- **Timer and Counter:** Timer and counter are two important features of embedded systems. Timer is used in embedded systems for measuring time and creating time delays. Whereas, counters are used for counting events occurring external to the system
- **Interrupts:** In embedded systems, interrupts are the signals produced by the microprocessor to stop the currently executing code or programs. These signals are important in embedded systems to enable real-time responses.
- **Real-Time Operating System (RTOS):** Real-time operating systems are specially design operating systems for processing data and events in real-time. These operating systems are programmed to complete tasks within a given time constraint or based on the demand of the event. Hence, these are also known as **event-driven operating systems**. RTOS are widely used in embedded systems to perform tasks based on real-time events.

Modern Trends in Embedded systems

Embedded systems have now **become the integral parts of all digital smart devices**, ranging from a **simple digital watch to complex robotic systems**. The following points highlight the contribution of embedded systems in modern world.

- Embedded systems are making **real-time data processing more advanced and faster**. This is an essential need in complex systems like **drones and other aviation systems**.
- Embedded systems are the **soul of IoT devices** like wearables, smart appliances, etc.
- Embedded systems are being empowered with modern technologies like **artificial intelligence (AI) and machine learning (ML)**. These technologies provide **self-decision-making capabilities to the embedded systems**.
- Embedded systems are also being **equipped with Edge Computing** that **reduces the delay and bandwidth** by processing data closer to its source. This technology is very important in real-time applications.
- Embedded systems are providing with advanced networking technologies like **4G, 5G**, etc. for more **efficient data communication**.
- Embedded systems are being also **integrating with quantum computing** for complex problem solving, optimized data processing, enhanced security, etc.
- Embedded systems are enabling **more precise 3D printing and improving printing processes** through real-time monitoring.

Applications of Embedded Systems

Embedded systems are the key components of all smart devices or systems. Some of the very **common applications** of embedded systems across various fields are listed here.

- **Automobiles:** In modern cars and vehicles, the embedded systems are used to perform various functions such as safety, navigation, infotainment, cruise control, engine health monitoring, and much more.
- **Smartphones:** In a typical smartphone, tons of embedded systems are used. These systems are responsible for performing different functions, from touch screen sensing to signal transmission, camera control, voice recording, detecting peripherals, and debugging.
- **Industries:** Embedded systems are essential components of robotics and automation systems in industries. They are used for processing data from sensors connected across industrial machinery and produce action instructions for their smooth operations.
- **Medical Equipment:** The functioning of advanced medical equipment like heart monitors, ventilators, etc. is dependent on embedded systems. In these devices, embedded systems automate their operation and collect data from sensors and convert them into meaningful results, helping medical staff to avoid errors and interpret the patient's conditions accurately.
- **Wearables:** Embedded systems are also used in wearables like smartwatches, fitness bands, etc. These systems are entirely responsible for connecting these devices with other IoT devices like internet and smartphones.

The Life Cycle of Embedded System Designs

Embedded systems have a **finite lifespan** throughout which they **undergo different stages**, going from system **conception or birth to disposal and in many cases, rebirth**. We call this the *Life Cycle of an Embedded System*. **Five stages** can be identified in the cycle: **Conception or Birth, Design, Growth, Maturity, and Decline**. Below figure illustrates the sequence of these stages, and the critical phases that compose each stage.

An embedded design is first **conceived** by the identification of a **need to solve a particular problem**, or the **opportunity to apply embedded technology to an old problem**. Many embedded systems are conceived by the need of changing the way old problems are solved or the need of providing **cost-effective solutions** to them. Other systems are conceived with the problems themselves. Opportunity arises with the realization of efficient solutions enabled by the use of embedded

systems. In any case, the application where the design needs to be embedded dictates the specifications that lead to the design stage.

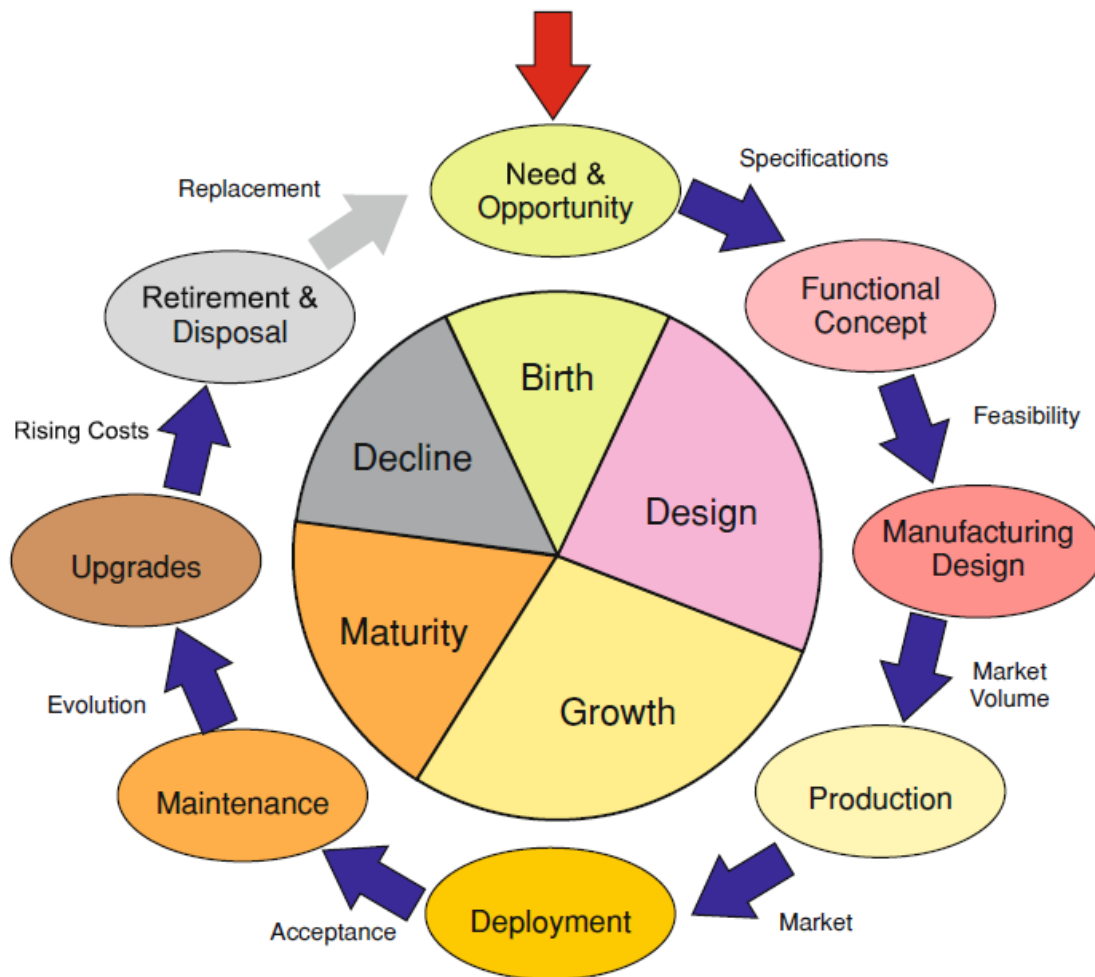


Figure: Life cycle of an embedded system

In the **design stage**, the system goes first through a phase of **functional design**, where **proof-of-concept prototypes** are developed. This phase allows to tune the system functionality to fit the application for which it was designed. A **feasibility analysis** that considers variables such as **product cost, market window, and component obsolescence** determines whether or not a **manufacturing and product design proceeds**. This second design phase establishes the way a prototype becomes a product and the logistics for its manufacturing. Design is by far the most costly stage in the life cycle of an embedded system, since most of the non-recurrent engineering costs arise here. The expected market volume guides the entrance into the growth stage.

The **growth stage** initiates with the **production** of the system to **supply an estimated market demand**. Production is scheduled to deploy the system at the beginning of its market window. The **deployment** phase involves **distribution**,

installation, and set-up of the system. The fitness of the system to its originating needs defines its acceptance, driving the embedded system into its mature stage.

In the **maturity stage**, the product is **kept functional and up to date**. This involves **providing technical support, periodic maintenance, and servicing**. As the application evolves, the system might require fitness to the changing application needs through software and/or hardware upgrades. The maturity stage is expected to allow running the product with minimal costs. As the system begins to age, maintenance costs begin to increase, signalling its transition into the decline stage.

In the **decline stage**, system **components become obsolete and difficult to obtain, driving the cost of maintenance, service, and upgrades** to levels that approach or exceed those of replacing the whole system at once. This is the point where the system **needs to be retired** from service and **disposed**. Product designs need to foresee this stage to devise ways in which recycling and reuse might become feasible, reducing the costs and impact of its disposal. In many cases, the **system replacement creates new needs and opportunities** to be satisfied by a new design, re-initiating the life cycle of a new system.

Embedded System Design Constraints

A **vast majority** of embedded systems **applications end up in the heart of mass-produced electronic applications**. Home appliances such as microwave ovens, toys, and dishwasher machines, automobile systems such as anti-lock brakes and airbag deployment mechanisms, and personal devices such as cellular phones and media players are only a few representative examples. These are systems with a **high cost sensitivity to the resources included in a design due to the high volumes** in which they are produced. Moreover, **designs need to be completed, manufactured and launched in time** to hit a market window to maximize product revenues. These **constraints shape the design** of embedded applications from beginning to end in their life cycle. Therefore, the list of constraints faced by designers at the moment of conceiving an embedded solution to a problem come from the different perspectives. The **most salient constraints** in the list include:

- **Functionality:** The system ability to perform the function it was designed for.
- **Cost:** The amount of resources needed to conceive, design, produce, maintain, and discard an embedded system.
- **Performance:** The system ability to perform its function on time.
- **Size:** Physical space taken by a system solution.
- **Power and Energy:** Energy required by a system to perform its function.

- **Time to Market:** The time it takes from system conception to deployment.
- **Maintainability:** System ability to be kept functional for the longest of its mature life.

Aside from functionality, the relevance of the rest of the constraints in the list changes from one system to another. In many cases, multiple constraints must be satisfied, even when these can be conflicting, leading to design trade-offs.

Advantages and Disadvantages of Embedded Systems

Advantages of Embedded Systems

- **High Performance and Efficiency:** Because they are designed to perform only one specific task, the hardware and software are perfectly tuned to work together without the "bloat" found in general-purpose computers.
- **Compact Size:** These systems are highly integrated, often fitting an entire computer's functionality onto a chip smaller than a coin. This allows them to be embedded in small devices like smartwatches or medical implants.
- **Low Power Consumption:** Most embedded systems are designed to run on batteries for long periods. They use low-power microcontrollers that can "sleep" when not in use.
- **Reliability and Stability:** Since the software (firmware) is fixed and doesn't change often, there are fewer chances for system crashes or "blue screens" compared to a PC.
- **Cost-Effectiveness:** Once the design is finalized, mass-producing the specialized hardware is significantly cheaper than using general-purpose components.
- **Real-Time Capabilities:** They can process data instantly, making them essential for safety-critical applications like automotive airbags or flight control systems.

Disadvantages of Embedded Systems

- **Limited Flexibility:** Once an embedded system is built, it cannot be easily repurposed. A microwave controller cannot be reprogrammed to run a digital camera.
- **Difficulty in Troubleshooting:** Because the components are so small and integrated, if one part of the circuit fails, it is usually impossible to repair. Often, the entire board must be replaced.
- **High Development Cost:** While mass production is cheap, the initial design, coding of firmware, and testing are expensive and time-consuming.

- **Resource Constraints:** They have very limited memory (RAM/ROM) and processing power. Developers must write extremely efficient code, as there is no room for "lazy" programming.
- **Lack of Scalability:** You cannot simply "add more RAM" to a microcontroller-based system. If the requirements of the project grow, you often have to redesign the entire hardware from scratch.
- **Maintenance and Updates:** Updating firmware can be difficult, especially for devices that are not connected to the internet. If a bug is found in the software after a product is sold, it can lead to expensive recalls.

IoT Embedded Systems

- The rapid progress of technology has blurred the lines between sectors. This is particularly evident in IoT (Internet of Things) embedded systems where the terms **IoT and embedded system have become mishmashes to mean the same thing.**
- But an **embedded system and IoT are two different concepts**, despite some similarities.
- Embedded systems have been around **since 1965** when a company called Autonetics developed a **tiny computer** that was **used as part of a missile guidance system.**
- An **IoT embedded system** is an **embedded system that also has connectivity to the internet** and can therefore **communicate with other IoT embedded systems.**

We are **surrounded by IoT embedded systems** such as:

- Set-top boxes
- Point-of-Sale (POS) terminals
- A wide variety of medical devices
- Any "smart" devices such as smart refrigerators, smart bicycles, fitness trackers, etc.
- Parking meters

What is IoT?

IoT stands for **Internet of Things** and refers to the **interconnected nature of devices** that we **use in our daily lives.** These days **"smart" devices can connect to the internet** and to other devices to **facilitate everyday actions.** For example, a **smart fridge can detect the items in it as well as expiry dates and provide information to the owner** about it. This is a typical use-case for IoT in everyday use.

What is Embedded System in IoT?

An IoT embedded system is an **embedded system that has internet connectivity**. Another word for what is IoT embedded system is a **"smart" device**. A touch screen and a keyboard are not necessary to define a device as an IoT embedded system, although these peripherals can also be attached.

Difference Between IoT and Embedded System

- The difference between IoT and embedded system in one word is **connectivity**. An **embedded system does not necessarily have internet connectivity**, but an **IoT embedded system always does**. A **pacemaker** is an **embedded system**. By **letting that device communicate to the outside world**, such as transmitting heart-rate data to a central database, it turns out into an **IoT embedded system**.
- The difference between an embedded system and IoT is that the **embedded system can be a subset of IoT**. IoT refers more to a class of devices that represent the newly connected world. But an embedded system refers very specifically to the hardware used in these devices.
- Embedded system before IoT were not able to connect to the internet, therefore IoT embedded systems generally encompass a broader range of devices and use cases.

Examples of non-IoT Embedded System Devices

It has become increasingly simpler to create IoT embedded systems. Although the **following examples might lately also have some IoT embedded system versions (such as a smart microwave)**, the following is a list of devices that have **typically not been used in an IoT embedded systems context**.

- Microwave ovens
- Washing machines
- Manufacturing robots
- Industrial robots and factory machines

However, modern warehouse robots are now typically created as an IoT embedded system that can communicate to a central dashboard via the internet and be managed using MDM services.

Even if there is no need for that particular device to be connected to the internet, opting for an IoT embedded system makes it possible for the device to receive IoT OTA updates so long as the proper operating system has been installed and the required infrastructure is given.

The Role / Use case of Embedded System in IoT

The embedded system in IoT devices is the hardware of the device. All **IoT devices are built on top of embedded systems**. Building IoT devices on the same hardware used for personal computers would be prohibitively expensive and is also unnecessary. The existence of embedded systems such as the Single Board Computers (SBC) Raspberry Pi, Rock Pi 4B, and others, means that IoT computing devices can be created on a small scale (one to 100,000 devices) while still keeping costs low. Here are **several use cases of very popular types of IoT embedded system devices**.

- **POS Terminals:** Point-of-Sale terminals and self-service checkout terminals are a common use of an IoT embedded system. These touch-enabled IoT embedded systems connect to payment gateways via the internet to process payments. These devices have a scanner attached to them, and can also receive input from a pin pad and card reader.
- **Vending Machines:** An IoT embedded system in the form of a vending machine can send messages alerting employees on the ground when items need to be refilled. The devices receive input from a coin slot that should be able to read the type of coin inserted. Further inputs include a touch screen and credit card reader.
- **Ticket Scanners:** Event venues can have a ticket scanner IoT embedded system in place that reads ticket information from visitors, compares it with a local or online database, and then engages a turnstile to let the visitor in.

These IoT embedded systems can be further programmed to send attendance data to an analytics platform, which can then process the data and provide detailed insight to event organizers.

Creating the Embedded System for IoT

It is possible for anyone to **create their own embedded system for IoT using just a few simple tools**. To create an IoT embedded system, you will **need the correct hardware and software**.

(i) Hardware for an IoT Embedded System

First and foremost, you will need the hardware for creating an embedded system for IoT. By then installing the appropriate software on this hardware and add needed peripherals, you can turn it into an IoT embedded system. There are numerous choices of hardware for an embedded system for IoT, including:

- Various **Raspberry Pi** models from Generation 3 and above.

- **Rock Pi 4B:** A single-board computer with similarities to the Raspberry Pi 4 but with a considerably enhanced CPU and GPU performance.
- **Intel NUC:** This is a powerful mini-PC with a highly customizable CPU, memory and storage setup. It is an excellent choice for a wide variety of use cases, especially those requiring a lot of power.

There are several other IoT embedded system hardware configurations you could use.

(ii) Software for an IoT Embedded System

Because an IoT embedded system connects to the **internet**, its **software must minimally be able to deal with these connections**. Usually, the choice of software for an IoT embedded system comes down to an **operating system such as Android or Linux**. **Installing** such software on an IoT embedded system **can be challenging** for several reasons:

- The **extremely varied hardware combinations** of these systems mean that OS support doesn't come "out of the box" for them. You have to **find versions of Android or Linux that have been specifically ported** to work on that particular hardware configuration, or develop it yourself.
- Ideally, the ported version of the software also includes support for **remote IoT embedded system updates and management**. To achieve this, the **ported version of the OS must also have a robust infrastructure** behind it that deals with creating new builds and pushing those builds out stably to IoT embedded devices.

Advantages of using Android for an Embedded System for IoT

Android has **several clear advantages over other operating systems** when used on an IoT embedded system.

- Android has a **high-quality user interface** and its **touch-screen support is familiar to users**.
- The **enormous popularity** of Android means that **hundreds of thousands of apps exist** which can be added to enhance that particular IoT embedded system.
- But probably the **most compelling reason** to choose a ported version of Android for your IoT embedded system is that Android **can be modified to support remote management** of devices as well as IoT OTA updates.

Embedded System vs Peripheral

To put it simply, an **embedded system** is the "**brain and soul**" inside a device, while a **peripheral** is the "**tool**" that connects to a main computer.

Core Definitions

- **Embedded System:** A self-contained "specialized computer." It has its own processor, memory, and software designed to perform one specific task. It is often hidden *inside* a larger machine (like the controller inside an elevator or a washing machine).
- **Peripheral:** An external device that "plugs into" a host computer (like a laptop or PC) to expand its capabilities. It cannot do much on its own; it needs the host computer to give it purpose.

Key Differences

Feature	Embedded System	Peripheral
Independence	Can often function entirely on its own (e.g., a digital watch).	Requires a host (PC/Laptop) to function.
Intelligence	Has its own "brain" (Microcontroller) and logic.	Usually acts as a "slave" to the host's CPU.
Purpose	To control a specific machine or process.	To provide Input, Output, or Storage for a PC.
Examples	Anti-lock Braking System (ABS), Microwave controller, Smart Thermostat.	Mouse, Keyboard, Printer, External Hard Drive.

The Relationship: The "Inside-Out" View

The most important thing to understand is that **most modern peripherals contain an embedded system**. Think of a **Printer**:

- **As a Peripheral:** You see it as a box sitting next to your computer that you use to print documents.
- **As an Embedded System:** Inside the printer, there is a complex embedded system. It has a processor that manages the ink nozzles, a motor controller to move the paper, and memory to store the document you just sent.

Why the Distinction Matters in Engineering

If you are a **Peripheral Designer**, you are focusing on how the device talks to the PC (USB protocols, drivers, etc.).

If you are an **Embedded System Engineer**, you are focusing on:

1. **Direct Hardware Control:** Moving the physical parts (motors, lasers).
2. **Firmware:** Writing the code that lives on the chip inside the device.
3. **Real-Time Response:** Ensuring the device reacts instantly to a button press or sensor trigger.